PWA Projet de Développement

Objectifs

L'objectif de ce projet est de développer l'application que vous avez imaginée et conçue comme une PWA, donc en utilisant les technologies web modernes. Vous devrez présenter la démo de l'application à la fin du semestre.

Vous avez un délai intermédiaire pour la liste des fonctionnalités à fournir pour le 27 Octobre sur Boost!Camp en PDF avec une présentation le 28 Octobre !

La date limite pour ce projet est le 10 décembre (code du projet et présentation à fournir sur Boost!Camp), c'est aussi la date de la présentation orale.

Instructions

A. Vous pouvez ou non mettre en œuvre toutes les fonctionnalités de votre prototype.
   a. Vous devez soumettre une liste de fonctionnalités que vous allez développer pour la V1 de votre PWA.
   b. Cette liste servira de base à votre évaluation le 10 décembre. Choisissez donc soigneusement ce que vous voulez mettre en œuvre (pas trop mais pas non plus une chose très simple).
   c. Vous devez soumettre la proposition en format PDF sur Boost!Camp avant le 27 Octobre à 23h58 !
   d. Le 28 octobre, vous devrez faire une brève présentation de vos fonctionnalités devant toute la classe.
   e. Les détails sont donnés dans le document spécifique sur Boost!Camp. Il s'agit d'une évaluation avec un coefficient de 1 (comme le quizz).

B. Dès le début du projet, vous devez nous fournir un accès Git (GitLab ou Github) au code de votre projet et au Figma ! (voir Annexe C pour les comptes).
C. Les frameworks web autorisés :
   a. Angular
   b. VueJS
   c. React
D. Pour l'interface utilisateur, vous pouvez utiliser le framework UI de votre choix (Tailwind, ou/et Ionic, ou autre).
E. L'application finale doit être une PWA, les instructions pour démarrer avec chaque framework sont dans l'Annexe A.
F. **En principe, la présence d'un backend n'est pas autorisée !** Si cela s'avère vraiment nécessaire, cette nécessité doit être documentée et validée avant le début du développement. Si un backend est développé, il ne sera pas pris en compte dans l'évaluation et il ne pourra pas être utilisé comme excuse si votre frontend ne fonctionne pas ! Voir l'Annexe B pour des possibles solutions.

G. Le design ne doit pas être parfait au pixel près par rapport au prototype, mais aussi proche que possible (laissez les détails à la fin et ne bloquez pas dessus).
H. L'utilisation de l'IA (agents, MCP, CLI, etc.) pour vous aider dans vos activités de programmation est autorisée, mais il vous incombe de pouvoir expliquer chaque ligne de votre code.
I. La possibilité de faire fonctionner l'**application sous iOS ou Android (Hybrid avec Capacitor) ou les deux est un plus et n'est pas nécessaire**. Cela n'entre absolument pas en ligne de compte dans l'évaluation.
J. N'attendez pas la dernière semaine pour commencer votre projet !
K. Vous pourrez poser des questions et recevoir de l'aide pour les problèmes que vous pourriez rencontrer pendant le cours consacré au projet PWA.

Présentation

Vous devrez présenter votre travail le 10 décembre.

Le but principal de cette présentation est de présenter votre projet PWA.

Vous aurez 30 minutes pour présenter votre projet et 15 minutes pour répondre à nos questions, soit **un total de 45 minutes**.

Votre présentation doit comporter les éléments suivants :

1. Un aperçu de votre produit/idée (un résumé de la partie marketing business).
2. L'architecture de votre application et les principales technologies que vous avez utilisées.
    a. Vous devriez nous montrer que l'application est une PWA.
    b. Quels outils pourriez-vous utiliser pour cela ?
    c. Quelles sont les exigences minimales pour qu'une application web soit une PWA ?
3. Une démo complète de votre application (nous apprécions les scénarios avec des histoires).
4. Une brève analyse de ce que vous avez appris et des problèmes/solutions que vous avez rencontrés.
5. Soyez prêt à répondre à quelques questions.

La présentation aura lieu dans votre salle de classe et nous vous enverrons l'horaire pour chacun d'entre vous.

Assurez-vous d'être présent et à l'heure pour le créneau horaire prévu et soyez présent au moins 5 minutes avant.

Évaluation

L'évaluation va de 1 à 6, 4 étant la note suffisante, avec un coefficient de 4.

Elle porte sur l'ensemble du semestre et se répartit selon les six critères suivants.

Critères et pondération

Critère	Poids	Ce que nous évaluons
Concept / Idée / Produit	20 %	La pertinence et la cohérence du produit imaginé, ainsi que la valeur qu'il apporte à ses utilisatrices et utilisateurs.
Décisions techniques et architecture	15 %	Les technologies retenues et votre capacité à justifier pourquoi celles-ci plutôt que d'autres.
Qualité du code produit	15 %	La lisibilité, la structure, la cohérence des conventions et l'absence de code mort ou dupliqué.
Maîtrise et défense du code	20 %	Votre capacité à expliquer, ligne à ligne, ce que fait votre code et pourquoi il est écrit ainsi.
Démarche et gestion du périmètre	10 %	La qualité de votre progression : itérations régulières, historique Git lisible, respect du périmètre annoncé le 28 octobre.
Présentation / Démo	20 %	La clarté du propos, la fluidité de la démonstration et la qualité du scénario présenté.

Ce à quoi vous devez vous attendre

Trois mécanismes concrets alimentent ces critères. Ils sont annoncés ici pour que vous puissiez vous y préparer.

1. Une revue de code pendant la soutenance

Sur les 45 minutes de soutenance, **5 à 10 minutes** seront consacrées à une revue de code. **Nous choisissons** deux fichiers ou deux commits dans votre dépôt, sans vous les communiquer à l'avance, et nous vous demandons de les expliquer.

Questions types :

- « À quoi sert cette fonction, et que se passe-t-il si on la supprime ? »
- « Pourquoi ce `useEffect` / cet `effect` / ce `watch` ici plutôt qu'ailleurs ? »
- « Ce commit modifie 400 lignes : raconte-nous ce que tu as changé et pourquoi. »
- « Où est géré le cas où la requête échoue ? »

Ne pas savoir répondre sur votre propre code coûte des points sur le critère **Maîtrise et défense du code**, quelle que soit la qualité du résultat visible à l'écran.

2. Un historique Git observé

L'accès Git nous est fourni dès le début du projet (instruction B) et **nous le consultons réellement**.

Nous attendons :

- des commits **réguliers et étalés** sur toute la durée du projet ;
- des messages de commit compréhensibles ;
- des unités de travail de taille raisonnable. Trois commits de 4 000 lignes déposés le 9 décembre sont en eux-mêmes une information, et pèsent sur le critère **Démarche et gestion du périmètre**.

3. Une déclaration de l'usage de l'IA

L'usage de l'IA reste **autorisé et encouragé** (instruction H). En contrepartie, votre `README.md` doit comporter une courte section indiquant **quels outils ont été utilisés et pour quelles parties du projet** (échafaudage de la couche de simulation, génération des données de test, composants d'interface, débogage, etc.).

Quelques lignes suffisent. Cette déclaration n'enlève aucun point : elle nous permet de cibler nos questions et vous protège du soupçon.

Point de vigilance

La complexité technique n'est plus valorisée pour elle-même. Une application au périmètre resserré, entièrement comprise et parfaitement défendue à l'oral obtiendra une meilleure note qu'une application riche en fonctionnalités que vous ne savez pas expliquer. **Choisir ce que vous ne construisez pas fait partie du travail évalué.**

Annexe A : Comment démarrer un projet PWA avec votre framework préféré

Afin d'éviter toute confusion concernant les instructions que vous pourriez trouver sur Internet, voici un récapitulatif des anciennes et nouvelles méthodes pour démarrer un projet PWA avec les trois frameworks JavaScript les plus utilisés : Vue, React et Angular.

Vue n'a plus de guide PWA de première main. L'ancienne documentation officielle, celle du plugin @vue/cli-plugin-pwa (cli.vuejs.org), est toujours en ligne mais Vue CLI est passé en mode maintenance : pour tout nouveau projet, l'équipe recommande create-vue et donc Vite. La référence à utiliser aujourd'hui est la documentation Vite PWA, qui propose un template Vue dédié (create @vite-pwa/pwa mon-app --template vue) et gère la génération du manifeste et du service worker via Workbox.

React est dans la même situation. La page officielle historique, Making a Progressive Web App de Create React App, est explicitement marquée comme obsolète : React a abandonné CRA en février 2025 et oriente vers un framework ou un outil de build comme Vite. Deux chemins officiels subsistent donc : la documentation Vite PWA avec le template react ou react-ts pour une application React classique, ou le guide first-party de Next.js, Progressive Web Apps, qui couvre le manifeste via app/manifest.ts, le service worker et les notifications push sans aucune dépendance externe.

Angular dispose de la seule documentation PWA officielle et maintenue en interne : le guide Service Workers & PWAs (vue d'ensemble et mise en route). La commande ng add @angular/pwa fait tout le travail : elle enregistre le service worker auprès des providers racine de l'application, installe les icônes nécessaires à l'installation de la PWA et génère le fichier de configuration ngsw-config.json qui définit les stratégies de cache.

Annexe B : Comment développer une PWA fonctionnelle sans backend

Simuler un back-end : garder le projet 100 % front-end

L'objectif est qu'un correcteur puisse cloner le dépôt, lancer npm install && npm run dev et obtenir une démo fonctionnelle **sans base de données ni serveur à déployer**.

- **Génération de données réalistes**, @faker-js/faker (v10.5.0, plus de 70 locales, dont fr). ⚠️ Utilisez bien ce fork communautaire : le paquet historique faker est déprécié et non maintenu. Pensez à faker.seed(12345) pour obtenir des données identiques à chaque exécution, indispensable pour des captures d'écran et des tests reproductibles.
- **Interception réseau**, MSW (Mock Service Worker), l'option recommandée pour les trois frameworks. Votre code appelle un vrai fetch('/api/users'), MSW intercepte la requête au niveau du réseau et renvoie une réponse simulée : aucune ligne de code applicatif à modifier le jour où le vrai back-end arrive. Il permet aussi de simuler les cas dégradés (erreur 500, latence, 429) et gère REST comme GraphQL. ⚠️ Vérifiez la cohabitation entre le service worker de MSW et celui de votre PWA (portées et ordre d'enregistrement), à tester tôt dans le projet.
- **Handlers avec état** : un simple tableau en mémoire dans les handlers MSW suffit à simuler un CRUD complet (POST qui ajoute, DELETE qui retire). Architecture conseillée en trois couches : fabriques de données (faker) → handlers → worker.
- **Alternatives acceptées** : json-server (API REST complète à partir d'un db.json, mais nécessite un second processus local) ; JSONPlaceholder ou ReqRes (API publiques en lecture seule, zéro configuration, utiles pour un prototype rapide). ⚠️ Mirage JS reste fonctionnel mais sa maintenance est incertaine, à éviter pour un projet destiné à durer.

Gestion de l'état

Il existe différentes façons de gérer l'état de votre application, allant d'un simple stockage sans persistance à une persistance plus complexe, entièrement structurée. C'est à vous de choisir celle qui vous convient, mais sachez que la difficulté augmente en fonction de la technique de persistance choisie. La plupart d'entre vous peuvent se contenter d'un simple “store” sans persistance.

- **Sans persistance** (état perdu au rechargement) : un store suffit, NgRx SignalStore ou le NgRx Store classique pour Angular, Pinia pour Vue, Zustand pour React. À noter : depuis les Signals, Angular n'impose plus de librairie externe pour des cas simples.
- **Persistance légère** (préférences, panier, session, quelques Ko) : gardez votre store et branchez un plugin de persistance. Pour Pinia, pinia-plugin-persistedstate suffit (persist: true sur le store, choix du storage, sélection des champs). Zustand propose son middleware persist en natif.
- **Persistance structurée** (listes volumineuses, données hors ligne, requêtes/index) : passez à IndexedDB avec Dexie. C'est le bon choix pour une PWA réellement offline-first, et il s'intègre bien avec Pinia comme avec les deux autres frameworks.
- Ou toute autre technologie répondant au besoin, à condition de justifier le choix.

Recours à un agent IA

- Un agent IA (Claude Code, Cowork, ou équivalent) est **autorisé et encouragé** pour échafauder cette couche de simulation : générer les types TypeScript du domaine, en déduire les fabriques faker, écrire les handlers MSW correspondants et produire un jeu de données de démonstration cohérent.
- Cas d'usage particulièrement efficace : fournir à l'agent une spécification OpenAPI ou un simple ensemble d'interfaces TypeScript, et lui demander de générer les handlers et la fausse donnée. C'est aujourd'hui la pratique courante plutôt que d'écrire les handlers à la main.
- Conditions : le code généré doit être relu, compris et défendable à l'oral. L'agent échafaude, vous restez responsable de l'architecture et des choix techniques.

Annexe C : Mes comptes pour me permettre d'accéder à vos projets Figma et Git

Ajoutez-les au projet avec un accès en lecture seule ; je n'ai pas besoin de modifier votre contenu ni votre code.

- Figma : mattia@dexio.ch
- Github : @mt-mattia
- GitLab : @mattia.gustarini.mt